

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set()
```

```
In [3]: df = pd.read_csv("student_exam_performance.csv")

df.head(-1)
```

Out[3]:

	student_id	age	gender	education_level	school_type	family_income	parent_e
0	STU_000001	20	Female	High School	Public	Middle	
1	STU_000002	17	Male	High School	Public	Middle	
2	STU_000003	18	Other	Undergraduate	Public	Middle	
3	STU_000004	20	Male	High School	Public	Middle	Hi
4	STU_000005	16	Female	High School	Private	High	
...	...	...	...	...	...	...	
99994	STU_099995	15	Male	High School	Public	Low	
99995	STU_099996	17	Male	Undergraduate	Public	Upper-Middle	
99996	STU_099997	14	Female	High School	Public	Upper-Middle	
99997	STU_099998	16	Female	High School	Public	Upper-Middle	Hi
99998	STU_099999	15	Female	High School	Private	High	

99999 rows × 44 columns



```
In [4]: df.dtypes
```

```
Out[4]: student_id      str
        age             int64
        gender          str
        education_level  str
        school_type      str
        family_income    str
        parent_education str
        urban_rural      str
        previous_exam_score float64
        previous_gpa      float64
        attendance_percentage float64
        assignment_completion_rate float64
        class_participation str
        study_hours_per_day float64
        self_study_hours  float64
        private_tuition    int64
        online_learning_hours float64
        study_consistency  str
        study_environment  str
        study_method       str
        revision_frequency str
        practice_tests_completed int64
        notes_quality      str
        sleep_hours        float64
        sleep_quality      str
        daily_screen_time  float64
        physical_activity_hours float64
        break_frequency    str
        stress_level       int64
        motivation_level   str
        internet_access    int64
        device_availability str
        educational_app_usage str
        online_course_hours float64
        exam_difficulty    str
        exam_preparation_days int64
        questions_attempted int64
        questions_correct  int64
        time_management_score float64
        exam_anxiety_level float64
        exam_score         float64
        performance_grade  str
        pass_status        str
        performance_level  str
        dtype: object
```

```
In [5]: df.head().style.format({'exam_score': '{:.2f}%'})
```

Out[5]:

	student_id	age	gender	education_level	school_type	family_income	parent_educ
0	STU_000001	20	Female	High School	Public	Middle	
1	STU_000002	17	Male	High School	Public	Middle	
2	STU_000003	18	Other	Undergraduate	Public	Middle	Ma
3	STU_000004	20	Male	High School	Public	Middle	High Sc
4	STU_000005	16	Female	High School	Private	High	Back



In [6]:

```
print(df.isna().sum())

df.duplicated()[df.duplicated()==True]
```

student_id	0
age	0
gender	0
education_level	0
school_type	0
family_income	0
parent_education	6526
urban_rural	0
previous_exam_score	0
previous_gpa	7793
attendance_percentage	9863
assignment_completion_rate	0
class_participation	0
study_hours_per_day	0
self_study_hours	0
private_tuition	0
online_learning_hours	0
study_consistency	0
study_environment	0
study_method	0
revision_frequency	0
practice_tests_completed	0
notes_quality	8407
sleep_hours	0
sleep_quality	6994
daily_screen_time	0
physical_activity_hours	0
break_frequency	0
stress_level	0
motivation_level	0
internet_access	0
device_availability	2965
educational_app_usage	0
online_course_hours	0
exam_difficulty	0
exam_preparation_days	0
questions_attempted	0
questions_correct	0
time_management_score	9613
exam_anxiety_level	0
exam_score	0
performance_grade	0
pass_status	0
performance_level	0

dtype: int64

Out[6]: Series([], dtype: bool)

```
In [7]: print("Shape ( row, colimns): ", df.shape, "\n")

print("number of rows: ",df.shape[0])
print("number of columns: ",df.shape[1])
```

Shape (row, colimns): (100000, 44)

number of rows: 100000
number of columns: 44

```
In [8]: df.describe(include='all')
```

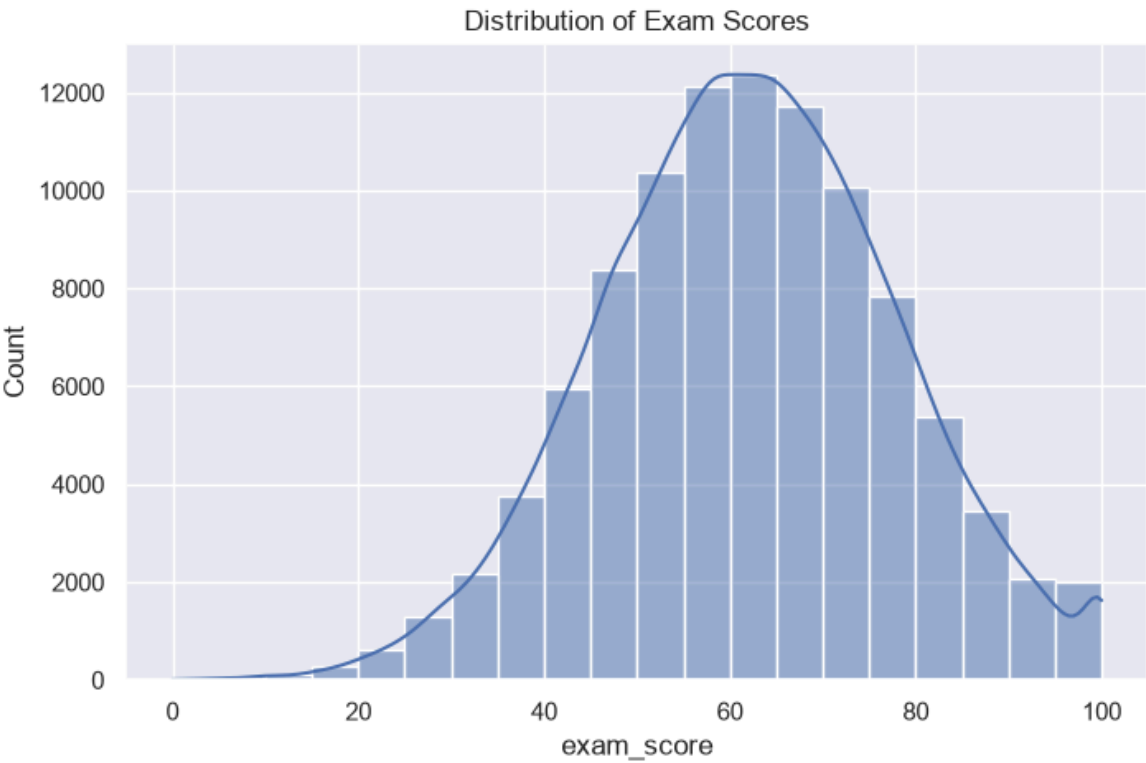
Out[8]:

	student_id	age	gender	education_level	school_type	family_income
count	100000	100000.000000	100000	100000	100000	100000
unique	100000	NaN	3	2	3	3
top	STU_000001	NaN	Male	High School	Public	Middle
freq	1	NaN	49130	59752	55105	30110
mean	NaN	17.005740	NaN	NaN	NaN	NaN
std	NaN	1.999562	NaN	NaN	NaN	NaN
min	NaN	14.000000	NaN	NaN	NaN	NaN
25%	NaN	15.000000	NaN	NaN	NaN	NaN
50%	NaN	17.000000	NaN	NaN	NaN	NaN
75%	NaN	19.000000	NaN	NaN	NaN	NaN
max	NaN	20.000000	NaN	NaN	NaN	NaN

11 rows × 44 columns

In [9]:

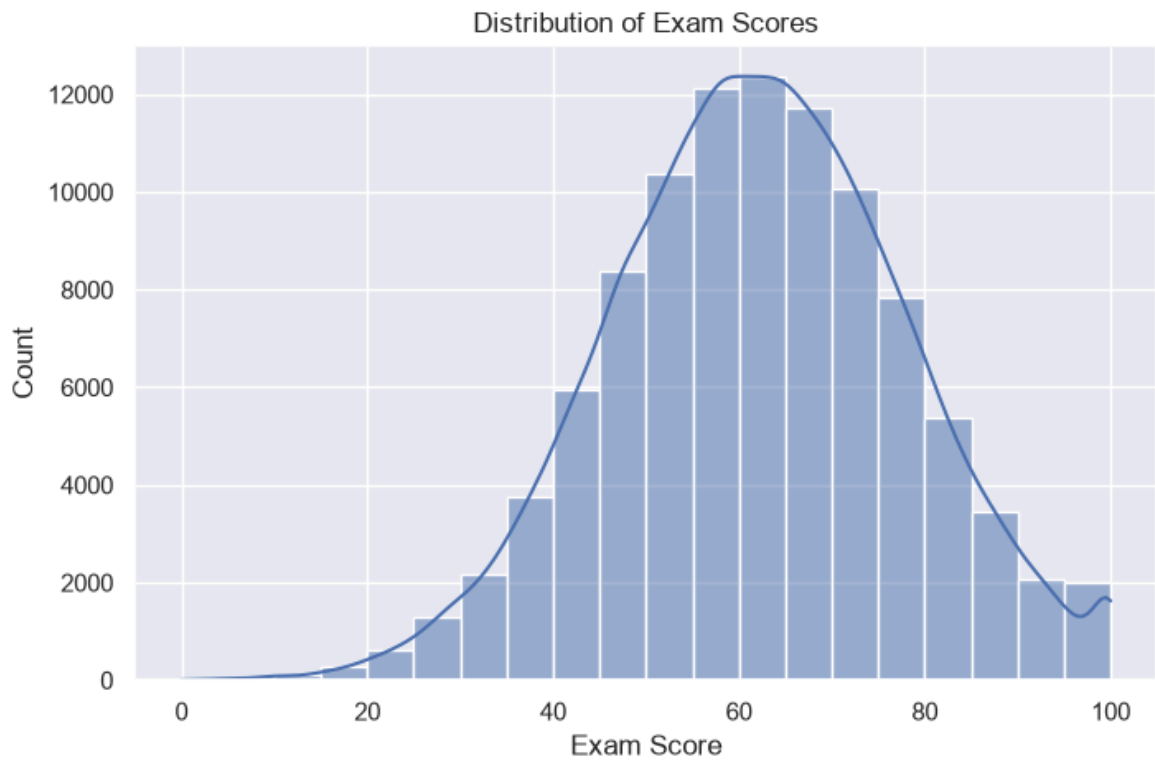
```
plt.figure(figsize=(8,5))
sns.histplot(df['exam_score'], bins=20, kde=True)
plt.title("Distribution of Exam Scores")
plt.show()
```



In [10]:

```
plt.figure(figsize=(8,5))
sns.histplot(df['exam_score'], bins=20, kde=True)
plt.title("Distribution of Exam Scores")
plt.xlabel("Exam Score")
```

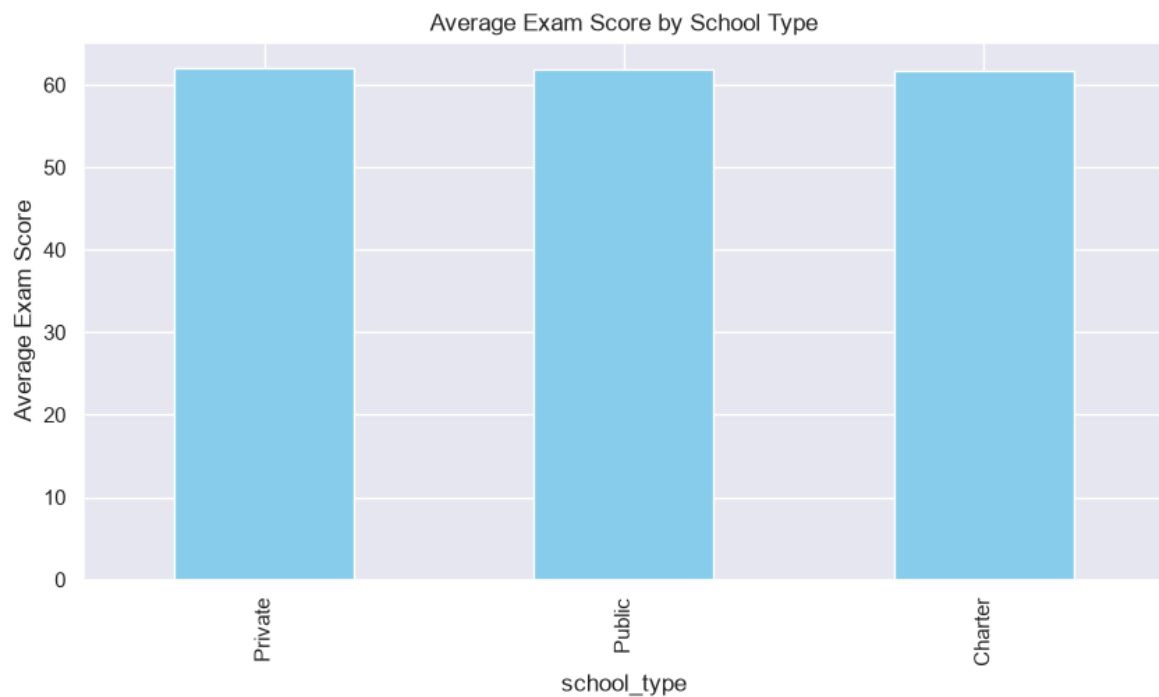
```
plt.ylabel("Count")
plt.show()
```



```
In [11]: school_performance = df.groupby('school_type')['exam_score'].mean().sort_values(

plt.figure(figsize=(10,5))
school_performance.plot(kind='bar' , color='skyblue')
plt.title("Average Exam Score by School Type")
plt.ylabel("Average Exam Score")
plt.show()

school_performance
```



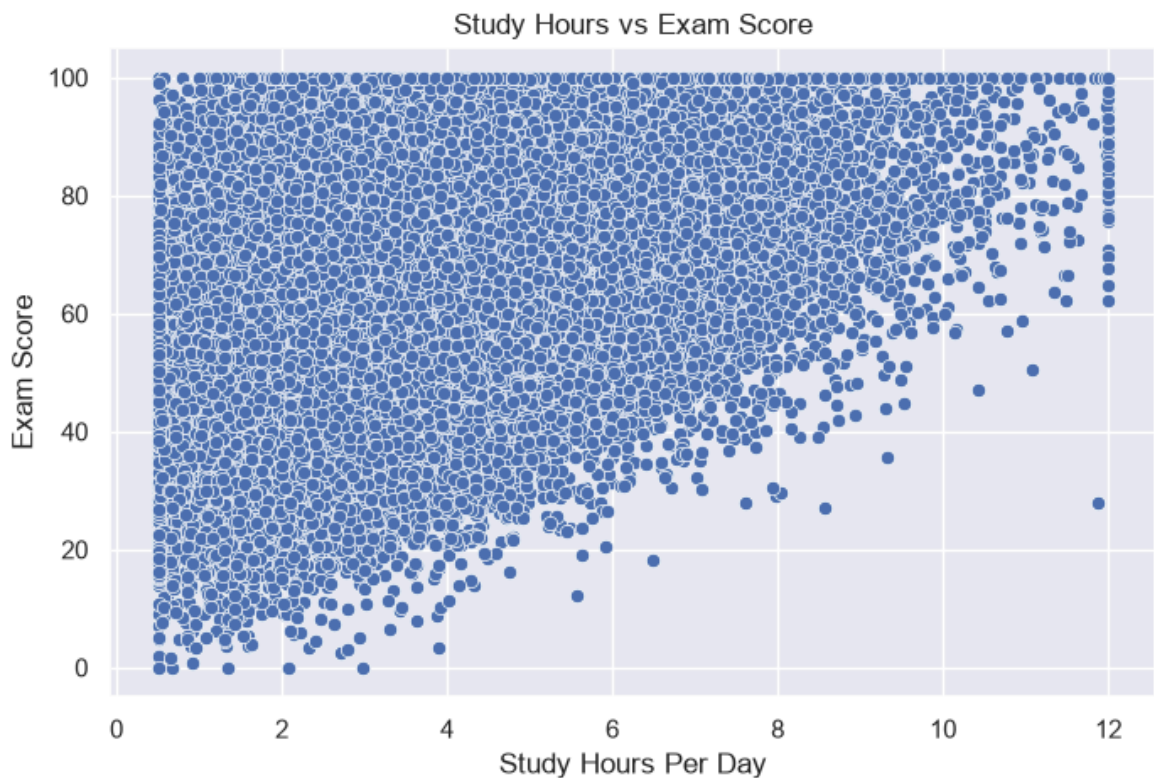
```
Out[11]: school_type
Private      62.058819
Public       61.938776
Charter      61.631952
Name: exam_score, dtype: float64
```

```
In [12]: study_method_scores = df.groupby('study_method')['exam_score'].mean().sort_value

study_method_scores.head(10)
```

```
Out[12]: study_method
Self-Reading      62.111513
Summarizing       61.997972
Practice Tests    61.994504
Flashcards        61.831542
Group Study       61.818061
Name: exam_score, dtype: float64
```

```
In [13]: plt.figure( figsize = (8,5))
sns.scatterplot( x = 'study_hours_per_day' , y = 'exam_score' , data = df)
plt.title("Study Hours vs Exam Score")
plt.xlabel("Study Hours Per Day")
plt.ylabel("Exam Score")
plt.show()
```



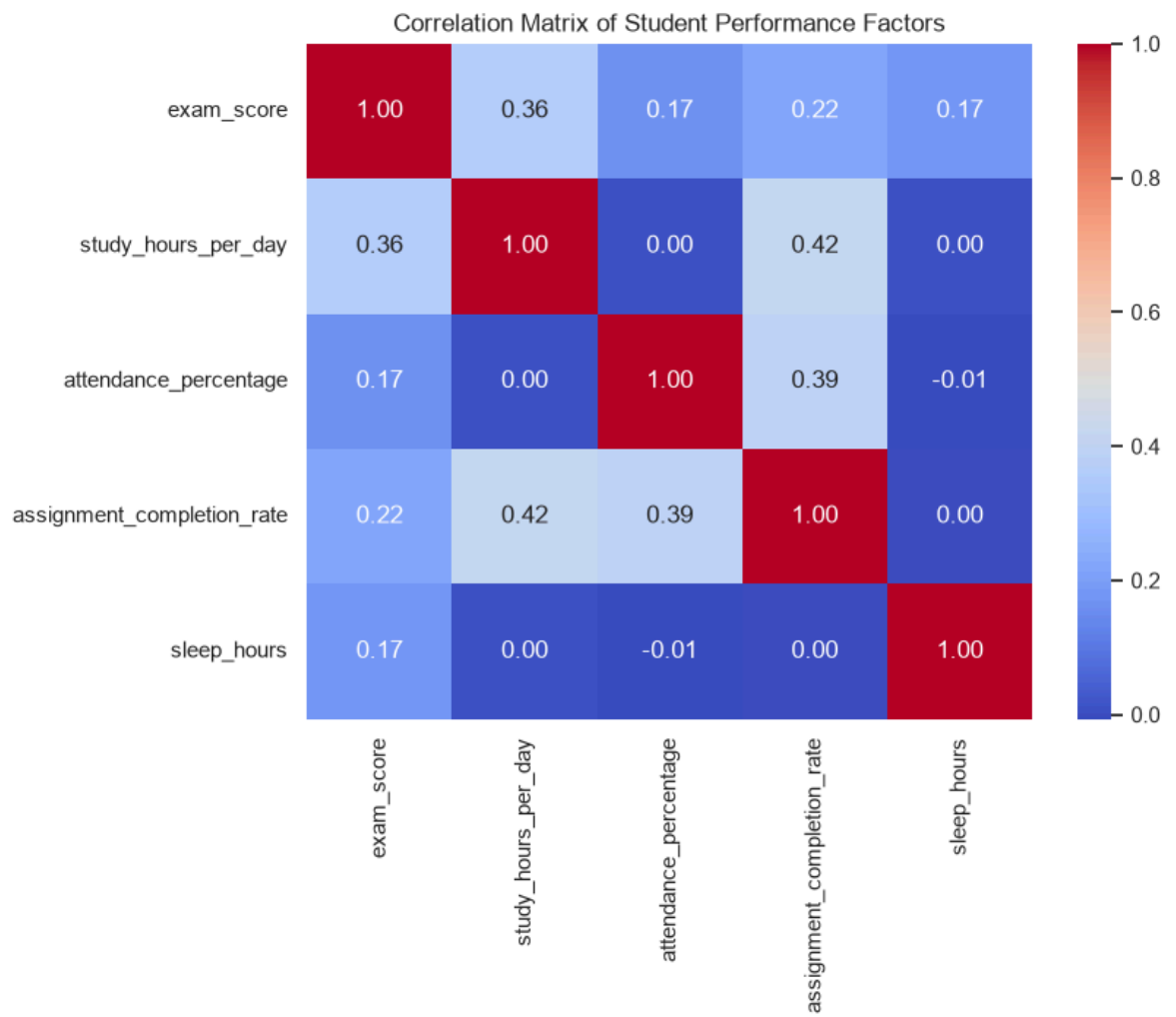
```
In [14]: plt.figure( figsize =(8,6))

numeric_cols= [ 'exam_score' , 'study_hours_per_day' , 'attendance_percentage' ,

sns.heatmap(df[numeric_cols].corr() , annot=True , cmap='coolwarm' , fmt=".2f")

plt.title("Correlation Matrix of Student Performance Factors")

plt.show()
```



In []: